

Bottleneck Paths Reduce to Deterministic Graphical Games and a Counterexample to a Claimed Linear-Time Algorithm

Egor Gorbachev  

ETH Zürich and Max Planck Institute for Informatics

Abstract

Chechik, Kaplan, Thorup, Zamir, and Zwick (STACS 2016) claimed a simple deterministic linear-time comparison-based algorithm for solving deterministic two-player, turn-based, zero-sum terminal-payoff games, also known as deterministic graphical games (DGGs). We give a counterexample to their algorithm.

We also give a deterministic linear-time reduction from the directed s - t bottleneck path (BP) problem to the DGG problem. Consequently, a linear-time comparison-based algorithm for computing the value of a designated start vertex in a DGG would yield a linear-time comparison-based algorithm for directed s - t BP. Whether directed s - t BP admits such an algorithm has remained open since Gabow and Tarjan gave their $\mathcal{O}(m \log^* n)$ -time algorithm. Thus, a positive resolution of the open question for DGGs would also resolve the corresponding open question for directed s - t BP.

Keywords and phrases directed bottleneck path, deterministic graphical games

Acknowledgements The author thanks László Kozma and Uri Zwick for helpful discussions.

1 Introduction

In the directed s - t bottleneck path (BP) problem, the input is a finite directed graph $G = (V, E)$ with $n := |V|$ vertices and $m := |E|$ edges, a weight function $w: E \rightarrow \Lambda$ into a totally ordered set Λ , and two distinguished vertices s and t . The goal is to find an s - t path whose largest edge weight is as small as possible. The problem appears in the work of Edmonds and Fulkerson [EF70]. In the comparison model, Gabow and Tarjan [GT88] obtained a deterministic $\mathcal{O}(m \log^* n)$ -time algorithm for directed BP (via their algorithm for directed bottleneck spanning trees). Chechik, Kaplan, Thorup, Zamir, and Zwick [CKT⁺16] later gave a slightly improved randomized comparison-based bound and a deterministic linear-time word-RAM algorithm. They left the existence of a linear-time comparison-based BP algorithm as an open problem.

A *deterministic graphical game* (DGG) is specified by a finite directed graph $G = (V, E)$, a disjoint partition $V = V_0 \cup V_1 \cup T$, a payoff function $p: T \rightarrow \Gamma$, and a designated start vertex $s \in V_0 \cup V_1$. Here, Γ is a totally ordered payoff set containing a distinguished draw payoff 0. Terminals have no outgoing edges, and every nonterminal vertex has at least one outgoing edge. A token starts at s . At a vertex $u \in V_i$, player i chooses an outgoing edge (u, v) and moves the token to v . If the token reaches a terminal t , the outcome is $p(t)$; an infinite play has outcome 0. Player 0, also known as MIN, tries to minimize the outcome, while player 1, also known as MAX, tries to maximize it. Solving the game from s means finding its value and optimal strategies for both players. The model was introduced by Washburn [Was90].

Andersson, Hansen, Miltersen, and Sørensen [AHMS10] gave a near-linear-time comparison-based algorithm for solving a DGG from a designated start vertex and asked whether the problem admits a linear-time comparison-based algorithm. Chechik, Kaplan, Thorup, Zamir, and Zwick [CKT⁺16] claimed to resolve this open question by giving a simple deterministic linear-time comparison-based algorithm.

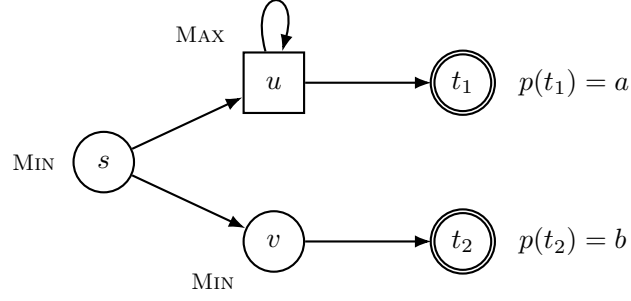
We propose a correction to this last result. First, we give a counterexample to the procedure of [CKT⁺16]. Second, we give a deterministic linear-time reduction from directed s - t BP to the problem of computing the value of a designated start vertex of a DGG instance. In particular, the DGG problem is at least as hard as directed s - t BP. These observations leave the following two questions open:

Does directed s - t BP admit a linear-time comparison-based algorithm?

Does DGG with a designated start vertex admit a linear-time comparison-based algorithm?

Our reduction shows that a positive answer to the second question would imply a positive answer to the first.

2 Bottleneck Paths Reduce to Deterministic Graphical Games



■ **Figure 1** A counterexample to the parallel procedure in the proof of Lemma 8 of Chechik et al. [CKT⁺16]. Circles represent MIN vertices, the square represents a MAX vertex, and double circles represent terminals.

2 The Claimed Algorithm and a Counterexample

Theorem 9 of Chechik et al. [CKT⁺16] claims a deterministic $\mathcal{O}(m)$ -time comparison-based algorithm that, given a DGG and a designated start vertex, finds its value and optimal strategies for both players. The key step in their algorithm is the following lemma. We restate the lemma and spell out its proof to make the issue self-contained.

■ **Lemma 2.1** (Lemma 8 of Chechik et al. [CKT⁺16]). *Let $G = (V, E)$ be a DGG with $V = V_0 \cup V_1 \cup T$, $T = \{t_1, t_2\}$, and $0 < p(t_1) < p(t_2)$. For $i \in \{1, 2\}$, let W_i be the set of vertices whose value is $p(t_i)$, and let*

$$E_i := \{(u, v) \in E : v \in W_i\}, \quad m_i := |E_i|.$$

If $W_1 \cup W_2 = V$ (that is, no vertex has value 0), then there is a deterministic algorithm for computing either W_1 or W_2 in $\mathcal{O}(\min\{m_1, m_2\})$ time.

Proof given by Chechik et al. Their Lemma 7 gives a backward-search algorithm for a game with a unique positive-payoff terminal. The search begins at the terminal and processes incoming edges. When processing (u, v) with v already discovered, it discovers u if u is a MAX vertex or if (u, v) is the last remaining outgoing edge of the MIN vertex u ; otherwise it removes (u, v) . Processing one incoming edge is called a basic step.

Run two copies of this search in parallel. In the first copy, add a self-loop to t_1 , so that t_1 is no longer a terminal; this copy is meant to construct W_2 . In the second copy, add a self-loop to t_2 and exchange the roles of the two players; this copy is meant to construct W_1 . Alternate basic steps between the two copies and stop both as soon as one copy finishes. Since the two searches are claimed to take $\mathcal{O}(m_2)$ and $\mathcal{O}(m_1)$ time, respectively, the parallel execution takes $\mathcal{O}(\min\{m_1, m_2\})$ time. ■

While the presented algorithm for computing W_2 is correct, the algorithm for computing W_1 does not work in general. Fix payoffs $0 < a < b$ and consider the game in Figure 1. Its vertices are $\{s, u, v, t_1, t_2\}$. The vertices s and v belong to MIN, the vertex u belongs to MAX, and t_1, t_2 are terminals with $p(t_1) = a$ and $p(t_2) = b$. The edges are

$$(s, u), \quad (s, v), \quad (u, u), \quad (u, t_1), \quad (v, t_2).$$

(If no self-loops are allowed, the self-loop of vertex u may be replaced by a cycle.)

The vertex values are immediate. We have $\text{val}(v) = b$ because the only move from v leads to t_2 . At u , MAX chooses between the payoff a at t_1 and the payoff 0 obtained by following the self-loop forever; hence $\text{val}(u) = a$. Finally, MIN chooses the smaller of the values at u and v , so $\text{val}(s) = a$. Therefore $W_1 = \{s, u, t_1\}$ and $W_2 = \{v, t_2\}$, and the hypothesis $W_1 \cup W_2 = V$ of Lemma 2.1 is satisfied.

Consider now the two searches in the claimed proof. The first search adds a self-loop at t_1 and searches backward from t_2 . It discovers v , since (v, t_2) is the last outgoing edge of the MIN vertex v . It does not discover s , since s still has the outgoing edge (s, u) . Thus this search computes the correct set $W_2 = \{v, t_2\}$.

The second search adds a self-loop at t_2 and exchanges the players. It starts from t_1 and processes the only incoming edge (u, t_1) . After the player exchange, u is a MIN vertex, and (u, t_1) is not its last outgoing edge because the self-loop (u, u) remains. The search therefore removes (u, t_1) instead of discovering u . Its worklist is now empty, so it terminates and returns $W'_1 = \{t_1\}$, rather than the true set $W_1 = \{s, u, t_1\}$. If this search receives the first basic step, the parallel algorithm stops immediately with this incorrect answer. If the W_2 search receives the first step, it only discovers v ; the following W_1 step still terminates with W'_1 .

3 A Linear-Time Reduction from BP to the DGG Problem

Let $(G = (V, E), w, s, t)$ be an instance of directed s - t BP, where $w: E \rightarrow \Lambda$ and Λ is totally ordered. Write

$$\text{bp}_G(s, t) := \min_{P: s \rightsquigarrow t} \max_{e \in P} w(e).$$

We extend Λ by two fresh values, denoted $\perp$ and $\mathbf{0}$, ordered as

$$\perp < \lambda < \mathbf{0} \quad \text{for every } \lambda \in \Lambda. \quad (1)$$

The fresh value $\mathbf{0}$ is the draw payoff assigned to an infinite play.

If the original BP weights are real and one insists that all game payoffs be real numbers, let $M := \max_{e \in E} w(e)$ and replace every weight by

$$\bar{w}(e) := w(e) - M - 1 < 0.$$

This translation preserves every optimal path and shifts the bottleneck value by $-(M + 1)$. Setting $\perp := \min_{e \in E} \bar{w}(e) - 1$ then realizes (1) over the reals, with the usual draw payoff 0; the original bottleneck value is recovered by adding $M + 1$.

We first delete every edge leaving t and discard every vertex that is not both reachable from s and able to reach t . Two graph searches perform this preprocessing in $\mathcal{O}(n + m)$ time. The preprocessing does not change the bottleneck value, and every remaining vertex other than t has an outgoing edge in the new graph. For notational simplicity, continue to call the resulting instance $G = (V, E)$.

For every edge $e \in E$, create a vertex x_e and a terminal z_e . The vertices of the game $\mathcal{G}(G)$ are

$$V \cup X \cup Z, \quad X := \{x_e : e \in E\}, \quad Z := \{z_e : e \in E\}.$$

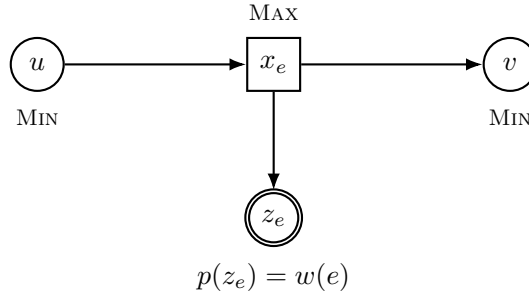
The vertex t and all vertices in Z are terminals. Every vertex in $V \setminus \{t\}$ belongs to MIN, and every vertex in X belongs to MAX. For each $e = (u, v) \in E$, add the three edges

$$(u, x_e), \quad (x_e, v), \quad (x_e, z_e)$$

to the game. Finally, set $p(t) = \perp$ and $p(z_e) = w(e)$. An infinite play has payoff $\mathbf{0}$, which is larger than every edge weight. The edge gadget is shown in Figure 2. The game has at most $n + 2m$ vertices and $3m$ edges.

■ **Theorem 3.1.** *The value of s in $\mathcal{G}(G)$ is $\text{bp}_G(s, t)$.*

4 Bottleneck Paths Reduce to Deterministic Graphical Games



■ **Figure 2** The DGG gadget replacing an edge $e = (u, v)$.

Proof. Write $B := \text{bp}_G(s, t)$. We prove matching upper and lower bounds on the game value.

For the upper bound, fix a simple s – t path P whose bottleneck is B . At each original vertex on P , MIN chooses the vertex x_e belonging to the next edge e of P . At x_e , MAX can stop at z_e , obtaining $w(e) \leq B$, or continue to the next original vertex. If MAX always continues, the play reaches t and obtains $\perp < B$. Hence this MIN strategy guarantees an outcome at most B , so $\text{val}_{\mathcal{G}(G)}(s) \leq B$.

For the lower bound, let MAX stop at z_e whenever $w(e) \geq B$, and otherwise continue to the end-vertex of e . A play that stops has payoff at least B , and an infinite play has payoff $\mathbf{0} > B$. If a play reached t without stopping, its projection in G would be an s – t walk using only edges of weight strictly smaller than B . Removing cycles would give an s – t path with bottleneck smaller than B , a contradiction. Thus MAX guarantees an outcome at least B , and $\text{val}_{\mathcal{G}(G)}(s) \geq B$. ■

■ **Corollary 3.2.** *If the value of a designated start vertex in a DGG can be computed (deterministically) in linear time in the comparison model, then directed s – t BP can be solved (deterministically) in $\mathcal{O}(n + m)$ time in the comparison model.*

Proof. Apply Theorem 3.1. The game $\mathcal{G}$ can be constructed in linear time. If an optimal path rather than only its value is required, perform a graph search in the subgraph consisting of the edges whose weight is at most $\text{bp}_G(s, t)$. ■

AI Disclosure. Large language models were used to prepare an initial draft from proof notes written by the author and to assist with figure creation. The author subsequently revised and verified the manuscript and takes full responsibility for its contents. All mathematical ideas and results are the author’s own.

Bibliography

- AHMS10** Daniel Andersson, Kristoffer Arnsfelt Hansen, Peter Bro Miltersen, and Troels Bjerre Sørensen. Deterministic graphical games revisited. *Journal of Logic and Computation*, 22(2):165–178, February 2010. doi:10.1093/logcom/exq001.
- CKT⁺16** Shiri Chechik, Haim Kaplan, Mikkel Thorup, Or Zamir, and Uri Zwick. Bottleneck paths and trees and deterministic graphical games. In *33rd Symposium on Theoretical Aspects of Computer Science (STACS 2016)*, volume 47, pages 27:1–27:13, 2016. doi:10.4230/LIPICS.STACS.2016.27.
- EF70** Jack Edmonds and Delbert R. Fulkerson. Bottleneck extrema. *Journal of Combinatorial Theory*, 8(3):299–306, April 1970. doi:10.1016/s0021-9800(70)80083-7.
- GT88** Harold N. Gabow and Robert E. Tarjan. Algorithms for two bottleneck optimization problems. *Journal of Algorithms*, 9(3):411–417, September 1988. doi:10.1016/0196-6774(88)90031-4.
- Was90** Alan Washburn. Deterministic graphical games. *Journal of Mathematical Analysis and Applications*, 153(1):84–96, November 1990. doi:10.1016/0022-247x(90)90267-j.